



Deriving a minimum set of viewpoints for maximum coverage over any given digital elevation model data

Xuan Shi & Bowei Xue

To cite this article: Xuan Shi & Bowei Xue (2016) Deriving a minimum set of viewpoints for maximum coverage over any given digital elevation model data, International Journal of Digital Earth, 9:12, 1153-1167, DOI: [10.1080/17538947.2016.1207718](https://doi.org/10.1080/17538947.2016.1207718)

To link to this article: <https://doi.org/10.1080/17538947.2016.1207718>



Published online: 20 Jul 2016.



Submit your article to this journal [↗](#)



Article views: 900



View related articles [↗](#)



View Crossmark data [↗](#)



Citing articles: 4 View citing articles [↗](#)



Deriving a minimum set of viewpoints for maximum coverage over any given digital elevation model data

Xuan Shi and Bowei Xue

Department of Geoscience, University of Arkansas, Fayetteville, AR, USA

ABSTRACT

This paper introduces how to automatically derive a minimum set of viewpoints for maximum coverage over a large scale of digital terrain data. This is a typical data and computation-intensive research covering a series of geocomputation tasks that have not been implemented efficiently or optimally in prior works. This paper introduces a three-step computational solution to resolve the problem. For any given digital elevation model (DEM) data, automatic generation of control viewpoints is the first step through map algebra calculation and hydrological modeling approaches. For each viewpoint, the viewshed calculation then has to be implemented. The combined viewshed derived from the viewshed of all viewpoints establishes the maximum viewshed coverage of the given DEM. Finally, detecting the minimum set of viewpoints for the maximum coverage is a Non-deterministic Polynomial-time hard problem. The outcome of the computation has broader societal impacts since the research questions and solutions can be adapted into real-world application and decision-making practice, such as the distribution, optimization and management of telecommunication infrastructure and wildfire observation towers, and military tactics and operations dependent upon landscape and terrain features.

ARTICLE HISTORY

Received 23 February 2016

Accepted 26 June 2016

KEYWORDS

Minimum set; maximum coverage; viewshed; digital terrain; optimization

1. Introduction

Digital elevation model (DEM) is a numeric simulation and representation of the Earth's terrain surface and has been a critical component in defining the digital earth system (Grossner, Goodchild, and Clarke 2008; Goodchild et al. 2012). DEM can be modeled as a raster grid or a triangular irregular network (TIN). Among varieties of applications of DEM, identifying a minimum set of observational viewpoints that can cover the maximum area of a given terrain has high values in many applications, including civil engineering, infrastructure optimization and management, and military operations. If a minimum set of wildfire watchtower or telecommunication antenna can be identified to cover the maximum terrain area, the solution has obvious and broad socioeconomic benefit. In the battlefield, if the army troops enter an unfamiliar area, such a solution can help understand which strategic locations are critical in the operations. For snipers, it helps to identify a list of potential locations to access the targets.

When high-resolution DEM data are available, it is a challenge for decision-makers even just to derive a list of candidate locations from the digital terrain. Given the example of the DEM data used in this study, each tile of DEM could have more than 1,600,000,000 pixels. Intuitively it has been said that if one person stands at a high place, this person can see much farther ahead. For this reason, the first step of the automation is to derive all the local peaks on the digital terrain as the candidate

locations for the minimum set calculation. In reality, however, it may not be true because the line of sight (LOS) of the highest place could be obstructed by lower surrounded places. For this reason, such lower surrounded places may have larger visible coverage areas. Consequently, for each candidate location, its viewshed or visible areas have to be calculated first before a heuristic approach can be developed to derive the minimum set of viewpoints from the candidate locations that can cover the maximum area of the given DEM.

Such a research has not been studied thoroughly and successfully due to its intensive computation essence. When big data are utilized in the calculation, such a problem is time consuming and almost not resolvable because a single computer has limited memory and computing capability. Through the literature review, we found that less than 10 relevant papers (Goodchild and Lee 1989; Lee 1991; Franklin 2002; Franklin and Vogt 2004; Kim, Rana, and Wise 2004; Magalhaes, Andrade, and Franklin 2010; Magalhaes, Andrade, and Ferreira 2010, 2011; Bao et al. 2015) were published in the past decades on this important and unsolved problem, while some empirical studies discussed how to improve the distribution of watchtowers since the location of the watchtowers were arbitrarily determined (Pompa-García et al. 2010; Sakellariou et al. 2015). Such a fact implies the difficulty in the research context since few successful works were accomplished yet to generate any feasible and efficient solutions.

This paper will first review the prior works and current status of relevant research as well as the unresolved problems to help understand the research context and challenges. Our key methods and workflows to derive a minimum set of viewpoints for maximum coverage over digital terrain data will be discussed, followed by implementation details using the Razor cluster hosted at the Arkansas High Performance Computing Center (AHPCC). Razor has 126 nodes. Each node has dual Intel Xeon 6-core 2.93 GHz X5670 with 24 GB of memory and 1 TB of local disk. The efficiency, feasibility, and high quality of the outcome results will be justified by comparing the existing location and distribution of Federal Communication Commission (FCC) towers and antennas. Future endeavors will be discussed in the conclusion.

2. Research context

Goodchild and Lee (1989) discussed a visibility coverage problem in 1989. They tried to find a set of viewpoints such that every location on the terrain surface can be seen from at least one viewpoint. They discussed the problem and solution by using a sample TIN data that have 100 vertices. Similarly the problem in this research can be elaborated as that, for any DEM data, how can we automatically derive a minimum set of viewpoints that cover the maximum area of the given DEM data? Within this paper, we call this as the minimum set cover problem. Theoretically, the problem can be elaborated as given a set U of n elements, and a collection $S = \{S_1, S_2, \dots, S_m\}$ of m subsets of U such that the union of S equals U . The set cover problem is to identify the smallest subset of S whose union covers U . Such an optimization problem is NP-hard (i.e. Non-deterministic Polynomial-time hard) (Leeuwen 1990; Feige 1998; Leiserson et al. 2001), which means the time required to find the answer grows dramatically as the problem size increases. More formally, no polynomial solution is identified for such a set cover problem, while approximate solutions can be explored using heuristic strategies. The problem of finding the minimum set of viewpoints whose joint viewshed covers the maximum terrain area is one application of the minimum cover set problem. In this case, all pixels of a DEM constitute the set U , and S_i corresponds to visible pixels of a viewpoint.

In the prior works, when TIN files are used to model a terrain landscape, the quality of such digital data is not satisfactory since many key features such as prominent mountain peaks or ridges could not be represented by TIN nodes or edges appropriately (Kim, Rana, and Wise 2004). As a result, visibility calculation based on TIN files will generate more fuzzy results, while a tiny TIN file with 100 vertices could hardly generate any meaningful solution in response to the challenge to resolve real-world problems. Except Goodchild and Lee (1989) and Lee (1991), no other products were

generated by using TIN files. When high-resolution DEM data are used, the sensitive differentiation of elevation information can be utilized for accurate surface analytics and hydrological modeling. However, the minimum set cover problem has seldom been elaborated in prior arts, let alone any feasible and practical solutions to process large DEM data sets.

Franklin (2002), Franklin and Vogt (2004), Kim, Rana, and Wise (2004), Magalhaes, Andrade, and Franklin (2010), and Magalhaes, Andrade, Ferreira (2010, 2011) discussed an alternative minimum set cover problem over a small DEM data set with a resolution ranging from 30 meters to 90 meters covering a spatial extent from 1201×1201 pixels to 4804×4804 pixels. As the authors acknowledged, it was a challenge to derive the initial optimized set of observational viewpoints from the entire DEM data due to the intensive computation involved. As a result, their solution was to divide the DEM into 12×12 segments. In each segment, a 100-meter radius was applied on each pixel of the DEM for visibility calculation along the LOS over 10 randomly selected points. The ratio of visibility calculation for each pixel over these 10 randomly selected points was used to generate the seeds. Finally in each segment, 7 pixels that have the highest visibility ratio were selected as the seed viewpoints (i.e. a total of $144 \times 7 = 1008$ seed viewpoints in the case of a DEM with 1201×1201 pixels). Their minimum set was then derived from these 1008 points. Because such an initialization process is not optimized when 10 randomly selected points are used, their minimum set of viewpoints would not cover the maximum area of the DEM data set. When more numbers of randomly selected points are applied, a longer computational time is expected, although they tried to use a graphics processing unit (GPU) to accelerate the calculation.

Kim, Rana, and Wise (2004) compared three heuristics strategies to find a specific number of viewpoints (2–10 in the paper) that jointly cover the maximum amount of visible pixels. This is a maximum coverage problem without identifying the minimum set of viewpoints. They used a DEM with 500-meter resolution and 40×40 pixels, resampled from a DEM covering $20 \times 20 \text{ km}^2$ area with 50-meter resolution and 401×401 pixels, ‘which was too large for some of the programs used in the analysis’. If their solution could not handle a tiny data of 401×401 pixels, such a solution could hardly be useful for any real-world problems.

Bao et al. (2015) discussed a similar issue in order to minimize the construction cost and to maximize the monitoring coverage of forest fires. In their work, the authors applied a threshold of the maximal coverage with a budget in the minimum set calculation problem. For the viewshed calculation, they wrote a Python script to do the calculation in ArcGIS 10. To derive minimal cost with full coverage and maximum coverage with limited cost, they use the open-source module LP_Solve to complete the calculation. The DEM for their study area has a dimension of 131 columns and 106 rows for a total of 13,886 cells at a spatial resolution of 30 meters. The candidate viewpoints or the location of watchtowers is manually recognized and identified. The scalability of such a unique case is limited when large and higher resolution data are involved in the geocomputation.

While the key computational tasks and workflows are introduced in detail in the following sections, it has to be highlighted that viewshed and minimum set calculation could be time-consuming. In this pilot study, 4527 control viewpoints could be automatically retrieved from a DEM tile named as *Gilboa*. In the other case, 4106 control viewpoints could be automatically retrieved from the DEM tile of *Summersville*. When each DEM tile has more than 16 million pixels, for each control viewpoint, one minute is used in the viewshed calculation. If a single computer is used to complete the viewshed calculation, it needs about 3 days. In the case of minimum set calculation, when the goal is to achieve maximum coverage of the digital terrain, a single computer may need more than 16 days to complete the task over one tile of DEM. For this reason, 100 processors on an AHPCC cluster were deployed through message passing interface (MPI) to complete the proposed workflow and tasks in a reasonable time. Since the goal of this pilot study is to examine the solution, high-performance computing solutions through accelerator technologies remain as the future endeavors as summarized in the conclusion.

3. Data and study area

High-resolution geospatial data have become increasingly available along with an accelerating increase in data volume. Figure 1 displays the index map of the state-wide 3-meter resolution DEM data for West Virginia. Each tile is generated based on the USGS 7 1/2' quadrangle. For example, the tile of DEM for *Gilboa* has a dimension of 3646×4626 pixels, or a spatial extent of about $11 \text{ km} \times 14 \text{ km}$. A single tile of DEM has about 64.34 MB and the whole state-wide data set has 495 tiles of similar DEM data. The data are available and free for this research. On the right part of Figure 1, about 4527 viewpoints (i.e. the small red dots on top of the DEM) can be automatically detected from the *Gilboa* DEM tile via a series of geocomputation procedures. The total viewshed area of these 4500 viewpoints can cover about 99.5% of the entire DEM. The goal of this research is thus to find the minimum set of these 4527 viewpoints that can cover 99.5% of the entire DEM.

In addition, FCC antenna structures registration data are used for validation and comparison to exemplify that the proposed solution is superior in identifying fewer number of viewpoints that have the same coverage designated by existing FCC antennas.

4. Key computational tasks and workflows

Within this study, DEM and viewshed are raster data, while viewpoints are point features. For each pixel on a DEM grid, if it is visible to a viewpoint, the corresponding pixel on the viewshed grid is assigned a binary value of 1, otherwise 0. The *joint viewshed* is the sum of viewshed derived from all viewpoints. The offset height of the observer at a given viewpoint is the height above the ground, while this location is the starting point of an LOS. The target height is the height above the ground at the end of the LOS.

To achieve the goal of deriving a minimum set of viewpoints for maximum coverage over large-scale digital terrain data, three categories of geocomputation tasks have to be implemented. First of all, for any given DEM data, all potential control viewpoints will be extracted automatically through map algebra calculation and hydrological modeling approaches. Second, the viewshed calculation has to be implemented on each viewpoint to generate the joint viewshed of all viewpoints to establish the maximum viewshed coverage of the given DEM. The minimum set cover computation is the last step to derive the minimum set of viewpoints that have their joint viewshed equal to the maximum coverage derived in step two.

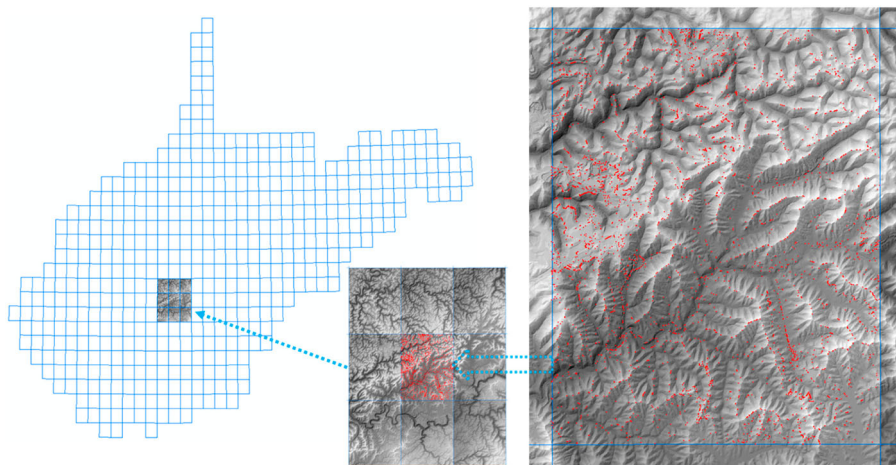


Figure 1. State-wide 3-meter resolution DEM data for West Virginia.

4.1. Automatic identification of control viewpoints over any given DEM data

Given a DEM data, normally we need to find such locations that have a relatively highest elevation (e.g. a peak location along a mountain ridge) in comparison to its surrounding area. Such locations will be used as the control viewpoints in the viewshed calculation. Identifying control viewpoints from a given DEM can be a complicated process by itself. One solution is to use an inverted DEM. In this case, the terrain is reversed. If the lowest points can be identified in the reversed DEM, the highest points can be identified in the original DEM. Within the reversed DEM, a sink is a cell with an undefined drainage direction, and no cells surrounding it are lower. The pour point is a boundary cell with the lowest elevation for the contributing area of a sink. If the sink were filled with water, this is the point where water would pour out. Thus, the sinks in the inverted DEM have to be filled to remove small imperfections in the data. In this process, the *z*-limit (i.e. the maximum elevation difference between a sink and its pour point to be filled) specifies the maximum depth of a sink that will be filled. In our prototyped project, the *z*-limit is set to be greater than 100 in generating the filled DEM. All sinks that are less than the *z*-limit, that is, lower than their lowest adjacent neighbor, will be filled to the height of their pour points. Due to the iterative nature of the fill function, it can be computation, memory, and I/O intensive. It can require up to four times the size of the input raster. The number of sinks found with the *z*-limit will determine the length of the processing time. The more sinks, the longer the processing time.

The algorithm of filling the sink is pretty straightforward. For a given DEM, a 3×3 window is applied over each cell of the DEM. If the value of the central cell is the smallest, it will be replaced by the smallest value of its eight surrounding cells. In case a *z*-limit is applied, all sinks that are less than the *z*-limit will be filled to the height of their lowest adjacent neighbor. The location of peaks can be identified through a raster calculation by using the criterion ' $\text{FilledDEM} - \text{InvertedDEM} > 0$ '. The output is a new raster (e.g. with a name as *peakLocationRaster*) with two values of 0 or 1. A second raster calculation is implemented by using the criterion ' $\text{peakLocationRaster} \times \text{OriginalDEM}$ '. Then only the elevation value for the peaks remains in the output grid if all 0 is set to be null. As a result, the vector point features can be derived as the control viewpoints for the given DEM. Due to the sensitivity of high-resolution DEM data, the point with the relatively highest elevation in comparison to its surrounding area may not necessarily be the mountain peak. In this case, the mean value of the given DEM could be used as a filter to remove some control viewpoints whose elevations are below the average.

4.2. Individual and joint viewshed calculation

Viewshed calculation could be time consuming. Major algorithms were reviewed in De Floriani and Magillo (2003). When the size of the DEM grows, the computation time would increase dramatically. This explains why prior works only dealt with tiny data (Franklin 2002; Franklin and Vogt 2004; Magalhaes, Andrade, and Franklin 2010; Magalhaes, Andrade, and Ferreira 2010, 2011; Kim, Rana, and Wise 2004). Given the example in Figure 1, if four tiles of DEM data are mosaicked, when about 14,000 viewpoints are utilized in viewshed calculation, ArcGIS – a well-known commercial geographic information system software product – could not complete the task in two or more weeks over more than 64 million pixels in the case an Intel i7-2760QM processor is applied. The algorithm discussed in Van Kreveld (1996) uses a sweep line to decide which pixels in the DEM should be examined to determine whether they obstruct other pixels that also intersect the line or be obstructed by others. Only pixels that intersect the line are extracted for computation. The line anchors one of its vertexes at the viewpoint and rotates 2π . An event is defined as the angle when the line enters a pixel, the line reaches the center of the pixel, or the line leaves the pixel. Thus, a pixel on a DEM data grid has three events corresponding to the three angles. Given a DEM of $n \times n$ in size, the balanced binary search tree, *T*, stores at most $2n$ nodes. The internal nodes store the maximum gradient of its subtree, making it possible to use previous computation results. As a

result, the sweep line algorithm does not need to perform LOS computation for each pixel and the time complexity reduces from $O(n^3)$ to $O(n^2 \log n)$. The implementation of the sweep line algorithm and the balanced binary search tree is discussed in Section 5.2.

Ferreira et al. (2014) proposed a parallel implementation of the sweep line viewshed algorithm. The parallel implementation takes advantage of shared memory and multi-thread to concurrently compute viewshed in multiple octants. Pixels within the viewshed radius are grouped into multiple octants. One thread handles one octant independently, and maintains its own event list and balanced binary search tree. OpenMP (Open Multi-Processing), an application programming interface (API) that supports multi-platform shared memory multiprocessing programs on most processor architectures and operating systems, is used to deploy multiple threads on one machine for speed-ups. The highest speed-up (12 times) is achieved using 16 threads on the 5000×5000 DEM. Ferreira et al. (2014) did not explicitly discuss possible performance issues when the numbers of pixels in different octants are unequal. Such load imbalance issue may dramatically lower down parallel speed-ups in extreme situations as discussed in Warn (2011), for example, when viewpoints reside at corners, or a large viewshed radius is adopted. In Warn (2011) several geospatial algorithms were parallelized using CPU multi-thread or a GPU. The sweep line algorithm is revised in data structure design and boosted with a single CPU via the multi-threading approach. In Zhao, Padmanabhan, and Wang (2013), although it was claimed that multiple GPUs were used to process multiple users' request via the so-called gateway interface, only a single GPU was used to process each individual request submitted from the web interface. In fact, Zhao, Padmanabhan, and Wang (2013) can only process the DEM with a dimension of about 2 miles by 2 miles, while the users have to wait for more than 20 minutes to get the results for such a tiny DEM. In fact, it was stated (Zhao, Padmanabhan, and Wang 2013) that 'a high degree of sequential dependencies in Van Kreveld's algorithm makes it less suitable to exploit parallelism'. Its claim of high performance computing over big data is questionable and problematic.

4.3. Deriving a minimum number of control viewpoints to cover a maximum area

Identifying the minimum number of control viewpoints that covers a maximum area is an NP-hard set cover or minimum cover problem. In this research, the optimization task is to find as few viewpoints as possible from a set of potential viewpoints that can cover the maximum of the terrain surface specified by the joint viewshed. Furthermore, this solution can resolve the minimum set problem on a user-defined ratio of the terrain surface.

Franklin (2002) discussed an approximation solution for this minimum cover set problem by iteratively selecting the viewpoint that mostly increases the visible area of the joint viewshed until the number of visible pixels in the joint viewshed reaches a specified threshold, although the joint viewshed coverage may exceed the coverage threshold when the last viewpoint is added to the solution set. In this study, we employed some new constraints in determining whether a viewpoint should be included into the minimum set or not. For example, the viewshed of a candidate viewpoint should not overlap too much with the viewshed generated by the other selected points, or its distance to other selected points should be larger than some value.

Figure 2 describes the algorithm and workflow for the minimum set calculation. Let P be the potential observer points, S be the set of selected points, and J_p and J_s are joint viewshed of P and S , respectively. V_i refers to the viewshed of viewpoint P_i . T_{cover} is the coverage threshold of the joint viewshed of selected viewpoints. T_d is the distance constraint. T_o is the viewshed overlap constraint. Consequently, the workflow for deriving the minimum set is implemented through multiple steps as the follows:

- (1) Initialize S as an empty set $\emptyset$ and compute J_p and J_s . In this case, the initial coverage of J_s is 0.
- (2) For each viewpoint P_i in P , compute its overlap fractions and Euclidean distances between viewpoints in S . Overlap fraction is defined as the overlap area of two viewshed data divided by the

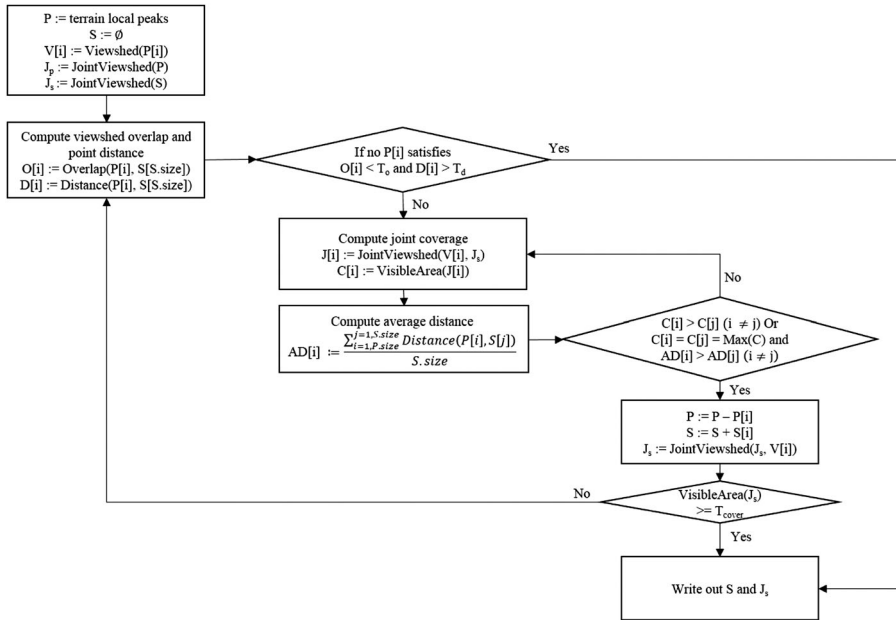


Figure 2. Workflow for deriving a minimum set of viewpoints for maximum coverage of DEM.

smaller viewshed coverage. If any overlap fraction is larger than T_o (e.g. 90%) or any distance is smaller than T_d (e.g. 250 meters), P_i will not be added to S . Therefore, if P_i overlaps too much with any points in S or is too close to any points in S , it should not be added to S . If no points in P satisfy the constraints, then go to step 4.

- (3) For each viewpoint P_i in P , compute its joint viewshed with J_s and its average distance to points in S . The viewpoint in P that has the maximum joint coverage will be added to S and removed from P . If two points have the same joint coverage, the one with the larger average distance will be added to S . J_s is then updated by setting it as the joint viewshed of the selected viewpoint and S . If the coverage of J_s is smaller than T_{cover} , then go to step 2.
- (4) Output S and J_s . The outcome of S will be a text file containing four columns, including the point ID, x , y , and the elevation of the corresponding pixel in the DEM grid. The joint viewshed raster J_s is generated in band interleaved by line (BIL) format.

5. Implementation of the workflow

The key computational tasks and workflows discussed in Section 4 have to be implemented over a computer cluster since the solution needs a large volume of storage, memory, and computing resources to complete the calculation.

5.1. Automatic derivation of potential control viewpoints

This task can be accomplished quickly without the need for parallelization. The following map algebra calculation procedures are deployed to derive the potential control viewpoints for any given DEM data.

- (1) Read DEM data;
- (2) Complete the inverse DEM calculation;

- (3) Complete the fill sink calculation over the inverse DEM. Single cell pits are pixels lower than all the eight neighbors. Pits are set as the lowest value of the eight surrounding pixels. After single cell pits have been filled, the implementation identifies and fills sinks using the algorithm described in Planchon and Darboux (2002);
- (4) Complete the minus calculation. Such a pixel level calculation is implemented between the filled inverse DEM and the inverse DEM. The calculation result is a raster with binary values. Pixels having unequal values in the two input raster data are assigned the value of 1 in the output grid;
- (5) Complete region group calculation. Pixels that have the value of 1 and are spatially adjacent in four directions are grouped together. The output result is a raster with background value as 0, while pixels that are grouped together will share the group ID as the pixel value;
- (6) Complete zonal max calculation. The output of region group is used as the filter for zonal statistics on the input DEM. The maximum value of all pixels in the DEM located in the same zone is assigned to the corresponding pixels in the output raster;
- (7) Identify the pixels in the DEM that have the same values as the corresponding pixels in the output from zonal max calculation. The output product is a binary raster, in which pixels that have a value of 1 indicate the location of the peaks in the DEM; and
- (8) Complete the raster to vector conversion calculation and write the output result as a text file, containing the information of point ID, x , y , and elevation of the control viewpoints, that is, peaks derived from the last step.

5.2. Viewshed calculation

The sweep line algorithm discussed in Van Kreveld (1996) is used for generating viewshed in this study. Warn (2011) implemented this algorithm by designing and employing a self-balanced binary search tree to store the maximum slopes that have been calculated or reuse the information when needed. Considering a large number of viewpoints were automatically identified, MPI is employed to simultaneously complete multiple viewshed calculation on a computer cluster hosted at the AHPCC. Each compute node has two Intel six-core Xeon X5670 'Westmere' 2.93 GHz processors.

The implementation of the sweep line algorithm was discussed by Warn (2011) in detail. The input DEM is first partitioned into eight octants with the viewpoint as the origin. The first octant contains pixels whose relative angle lies between 0 and 0.25π . The range of angles of the other octants increases by a step of 0.25π . The implementation assumes that the sweep line rotates in the counter-clockwise direction.

The solution then sorts the pixels into an event list based on their relative angle to the viewpoint. After the pixels have been sorted, the implementation examines visibility starting from the first pixel in the event list. For a pixel under processing, the implementation calculates its height slope to the viewpoint and compares it with the maximum slope of pixels, which are currently intersected by the sweep line and reside in preceding positions in the event list. If the former is larger, the current pixel is visible to the viewpoint; otherwise, invisible.

A binary search tree is applied as a status structure that records the slopes of pixels. The status structure could be conceptually viewed as a named list. The names are distance to the viewpoint and elements are height slope to the viewpoint. Instead of using Euclidean distance, the implementation measures the distance between a pixel and the viewpoint by the difference of their rows or columns depending on which octant the pixel resides in.

As the sweep line rotates, elements in the named list are updated while the names remain the same. For example, pixel Xa resides in a later position to pixel Xb and they all locate in the first octant and their column index is the same. In this case, the height slope of Xa stored in the status structure would be replaced by the height slope of Xb when Xb is encountered by the sweep line. After all pixels in the event list have been processed, the algorithm terminates.

In the following pseudo-code, the event list E is a sequence of n pairs of column and row in the first octant, each of which corresponds to a pixel. G is the input DEM, $G(E_i)$ is the pixel value at location E_i . R is the output viewshed. S is the status structure. The viewpoint is denoted as V .

```

1: initialize status structure  $S$ , i.e., the binary search tree
2:  $j := 0$ 
3: for  $i$  in 0 to  $n-2$ 
4:    $(x, y) := E_i$ 
5:    $\text{slope} := \text{Prefix}(S, x - V.x)$ 
6:    $R(E_i) := \text{Visible}(V.z, \text{slope})$ 
7:   if  $V, E_i, E_{i+1}$  are collinear
8:      $S[x - V.x] := (G(E_i) - V.z)/(x - V.x)$ 
9:   else
10:    while  $j \leq i$ 
11:       $(x, y) := E_j$ 
12:       $h := \text{Min}(G(E_j), G(x, y+1))$ 
13:       $S[x - V.x] := (h - V.z)/(x - V.x)$ 
14:       $j := j+1$ 
15:  $R(E_i) := \text{Visible}(S, G, E_{n-1})$ 

```

In line 5, function `Prefix` performs prefix scan on elements in preceding positions of $x - V.x$ in the status structure S . Thus, the maximum slope of pixels that may block E_i is extracted. Then line 6 compares the maximum slope with the height slope of E_i and determines its visibility from the viewpoint. Lines 7–14 update S , if E_i, E_{i+1} and V are collinear, then the height value at E_i is used directly for computing the height slope of E_i ; otherwise, lines 10–14 interpolate the height values and update S using the interpolated heights. The index j tracks the next pixel needs to be interpolated. In summary, the time complexity of the sweep line algorithm is $O(n^2 \lg n)$, given a $n * n$ grid, which is much better than the $O(n^3)$ of the R3 algorithm (Warn 2011).

Since each tile of the DEM used in this study has more than 16,800,000 pixels, viewshed calculation by the sweep line algorithm could consistently need one minute to accomplish when a serial computer program is applied on an Intel Xeon X5670 2.93 GHz processor. When potential control viewpoints are automatically derived, the total number of such viewpoints could be over 4000. As a result, viewshed calculation is time consuming when the viewshed of all potential viewpoints has to be calculated one by one.

In this study, AHPCC cluster is applied to complete the viewshed calculation over a large volume of viewpoints through MPI by employing 100 processors; each processor completes the viewshed calculation for a set of viewpoints by following a generic divide-and-conquer approach; that is, each processor will work on $n/100$ viewpoints, while one processor will handle the residual number of viewpoints. Obviously, the load balance and performance issues can be improved, though this research would like to first solve the computational problem.

5.3. Minimum set of control viewpoints for maximum coverage

The goal of the minimum set calculation is to identify the smallest set of viewpoints that can cover the maximum area of given DEM data. The coverage of the joint viewshed of the minimum set should be equal or greater than a specified coverage threshold. In this study, the threshold is the joint viewshed of all automatically derived viewpoints. Furthermore, the threshold is adjusted by different ratios of the joint viewshed.

The algorithm discussed in Franklin (2002) and Leiserson et al. (2001) is applied and adjusted in this study to iteratively select automatically derived viewpoints until the threshold criteria are satisfied. The viewpoints automatically derived from step one and their individual viewshed computed in step two are read into memory. To save memory space, bitmap data structure in C++ standard template library (STL) is used to hold the viewshed data in memory. Since each viewshed grid contains 0s and 1s, when an 8-bit bitmap data structure is applied, the required memory can be reduced four

times in comparison to using a 32-bit integer to capture the viewshed information. Considering more than 4000 viewshed data are used in the calculation and each has more than 16 million pixels, such a strategy is critical and efficient in the minimum set calculation.

After viewpoints and viewshed data have been loaded into memory, the implementation iteratively finds the point that mostly increases the joint viewshed coverage. In the first iteration, the solution set is initialized to an empty set first, and then the viewpoint with the largest viewshed coverage is added to the solution set. Starting from the second iteration, the distance and overlap constraints are applied. At the end of each iteration, the number of available potential viewpoints decreases by one. Once the coverage threshold is achieved, the iterative process terminates. The solution set and its joint viewshed are written out.

The pseudo-code below describes the minimum set selection process. Besides parameters discussed in Section 4.3 (e.g. P , S , J_s , J_p , T_d , T_o , T_{cover}), $meanDistance$ is a vector storing the average distances between points in P and S , and $coverage$ is a vector storing the joint viewshed coverage of every point in P and J_s . V_i is the viewshed of viewpoint P_i .

```

1:  $S := \emptyset$  // initialize set  $S$  to empty
2: while  $J_s < T_{cover}$ 
3:   for each  $P_i$  in  $P$ 
4:      $d[i] := distance(P_i, S[Size])$  // Euclidean distance
5:      $o[i] := overlapArea(P_i, S[Size])$  // overlap fraction
6:     if  $d[i] \geq T_d$  or  $o[i] \leq T_o$ 
7:        $meanDistance[i] := (meanDistance[i] * (S.size - 1) + d[i]) / S.size$ 
8:        $coverage[i] := number\ of\ visible\ pixels\ in\ J_s \cup V_i$ 
9:   select  $P_k$  such that  $coverage[k] = \max(coverage)$ ; if  $coverage[i] = coverage[j] = \max(coverage)$ , then select the one with the large mean distance.
10:   $J_s := J_s \cup V_k$ 
11:   $P := P - P_k$  // the size of  $P$  decreases
12:   $S := S + P_k$  // the size of  $S$  increases;  $P_k$  is the newly added point
13: return  $S$  and  $J_s$ 

```

Lines 2–11 iteratively add viewpoints to S until the threshold of joint viewshed coverage is reached. Lines 4 and 5 calculate distance and overlap fraction on P_i . Only if P_i satisfies the constraints, its joint viewshed with J_s is then computed. Line 9 finds the viewpoint which has the largest joint viewshed coverage and also satisfies the constraints.

The iterative process can be time consuming as it performs raster overlay calculation and counts viewshed coverage in each iteration. As the size of the potential set increases, the computation time would increase dramatically. In this study, AHPCC cluster is employed for parallelized minimum set calculation through MPI commands. When N processors are deployed, each viewshed grid is divided into N segments by rows. Each processor reads the corresponding indexed segment of all viewshed data. To count the coverage of a viewshed, each processor first counts the coverage of the segment of viewshed allocated to it. Then, MPI reduction is performed to sum up the coverage on single processors and broadcast the result over all processors. Each processor maintains one copy of a potential set of automatically generated viewpoints, the solution set as the output minimum set, and the joint viewshed of the solution set. Such data are synchronized in each iteration, but only processor 0 is responsible for writing out the solution set and its joint viewshed.

Calculating viewshed overlap areas and Euclidean distances between viewpoints are also computationally intensive. The two tasks can be accelerated using MPI. Since a single viewshed is distributed over all MPI processors, the implementation first counts the number of overlapped pixels separately on individual processors, then adds up the individual counts to get the total number of overlapped pixels. Thus, each viewpoint in P is processed sequentially, while calculating its viewshed overlap area is executed in parallel. When computing the Euclidean distances between the viewpoints in P and the newly added viewpoint in S , as shown in the above pseudo-code, the parallel strategy is similar to that for multiple viewshed calculation.

6. Result and discussion

The proposed workflow successfully derives the minimum set of control viewpoints for maximum coverage of the given terrain based on user-specified criteria. In this exploratory study, one DEM tile named *Summersville* was used to complete the calculation, while several other DEM tiles were tested as well. This tile of *Summersville* DEM has 3646×4626 pixels for a spatial extent of about $11 \text{ km} \times 14 \text{ km}$. Figure 3 displays the results of the minimum set calculation when the offset height of the observer is set to 1 foot. First of all, 4106 viewpoints are automatically derived from the calculation. The joint viewshed of these 4106 viewpoints can cover about 99.5% of this tile of DEM.

The minimum set is derived from the proposed workflow and has 1217 viewpoints to achieve the goal of maximum coverage of 100%, which means the selected minimum set of viewpoints can achieve the same coverage of 4106 viewpoints on the tile of *Summersville* DEM. Obviously such a minimum set still contains a large number of viewpoints because many single cells in the DEM grid can only be seen by one viewpoint. When the criteria of maximum coverage are changed, the number of minimum set can be reduced significantly. In Figure 3(b), 45 points can cover 90% of this tile of DEM. Consequently, when the maximum coverage is reduced to 80%, 70%, 60%, and 50%, the minimum set is reduced correspondingly to 22, 12, 6, and 4 viewpoints.

When 100 processors on the AHPCC cluster were deployed to complete the proposed workflow and tasks, the viewshed calculation over 4106 control viewpoints could be finished in about 42 minutes. In the case of minimum set calculation for the maximum coverage (100%), 240 minutes were used. When the maximum coverage is reduced to 90% or less, the calculation time could be reduced significantly since there is no need to repeat the iteration for a thousand times. Besides the tile of *Summersville* DEM, the proposed solutions were applied on other tiles of DEM. For example,

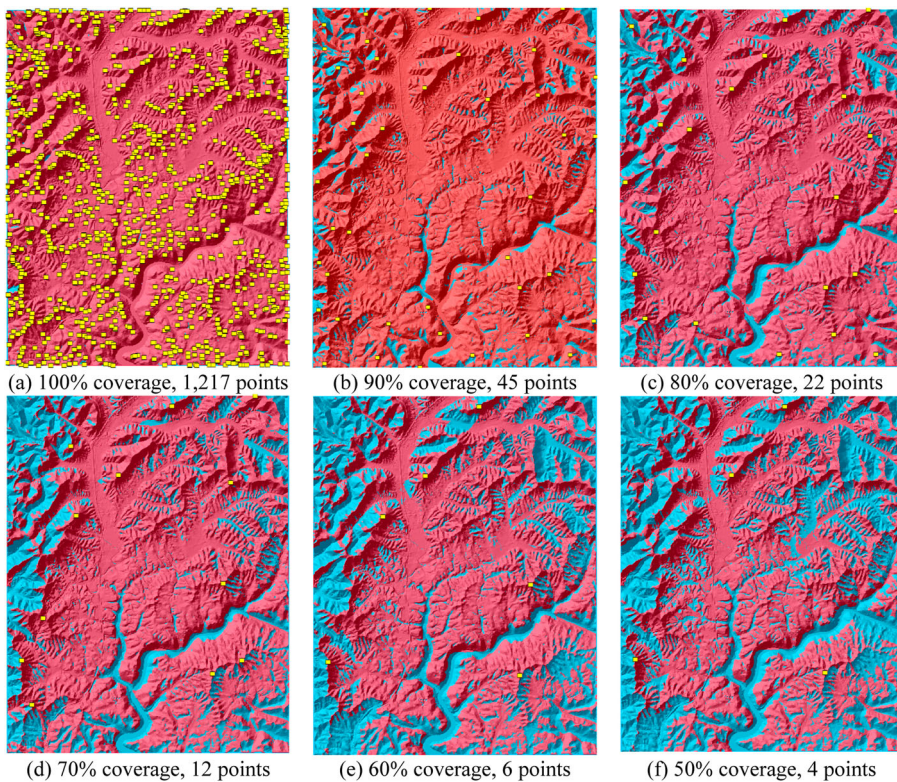


Figure 3. Minimum set of viewpoints for varied maximum coverage of DEM. (a) 100% coverage, 1217 points. (b) 90% coverage, 45 points. (c) 80% coverage, 22 points. (d) 70% coverage, 12 points. (e) 60% coverage, 6 points. (f) 50% coverage, 4 points.

4527 points were automatically derived from the tile of *Gilboa* DEM, while a minimum set of 1275 viewpoints can cover the same terrain surface as that for those 4527 points. In the case of *Swandale* DEM, a minimum set of 1550 points can have the same coverage by the original 2698 viewpoints that were automatically generated. The performance of the solution has been consistent depending on the number of viewpoints that were generated through the automated procedures. On average, the viewshed calculation needs about one minute to complete by one processor. If more number of viewpoints is generated through the automated procedures, the minimum set calculation would take longer time to achieve the maximum coverage.

The proposed solution has a significant value in real-world practice. Figure 4 displays the visibility coverage of current locations of FCC antenna at different offset heights in the same tile of DEM, in which 10 FCC antennas are installed in this area. In comparison to the minimum set of viewpoints derived from this optimized solution, even when the height of the antenna is set to 60 feet, these 10 FCC antennas can only cover 49.74% of this area. If the location is determined by the proposed solution, only 4 towers/antennas are required and can cover 50% of this tile of DEM even when the offset height is set to 1 foot. This means such a solution can save lots of money for the FCC. Besides the potential of enormous economic benefit, the comprehensive research will enable the analysis of the accessibility, disparity, and quality of telecommunication services even for an individual location or area. Meanwhile, such a research has a significant value in military operations, geospatial intelligence analytics, and homeland security applications.

Since the minimum set is selected from those automatically derived viewpoints, such an initial set generated from the automatic computing process is critical to the quality and quantity of the minimum set. For example, as briefly discussed in Section 2, the so-called SITE algorithm described in Franklin (2002) also consists of three steps to identify potential viewpoints, compute the viewshed of each viewpoint, and generate the solution set. However, SITE applies different approaches to identify the potential viewpoints in the first step and compute the viewshed in step two. Moreover, SITE does not take into account the distance and viewshed overlap constraints. In comparison to our solution, the minimum set derived by SITE contains much more viewpoints to cover the same area of a DEM.

Figure 5 displays the initial set of viewpoint generated by different approaches. The data are a subset of Summersville DEM containing 2000 rows and 2000 columns. The viewpoints in Figure 5(a) are local peaks distributed along mountain ridges derived by the proposed workflow discussed in Sections 4.1 and 5.1. Figure 5(b) displays the initial set of viewpoints generated by the approaches of SITE discussed in Franklin (2002). Consequently, Figure 6 displays the different results of the minimum set calculation based on these two different approaches.

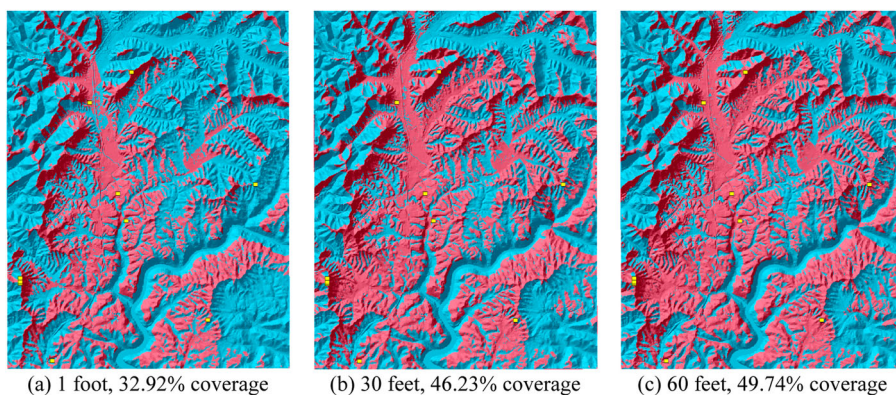


Figure 4. The visibility coverage of 10 locations of FCC antennas at different offset heights. (a) 1 foot, 32.92% coverage. (b) 30 feet, 46.23% coverage. (c) 60 feet, 49.74% coverage.

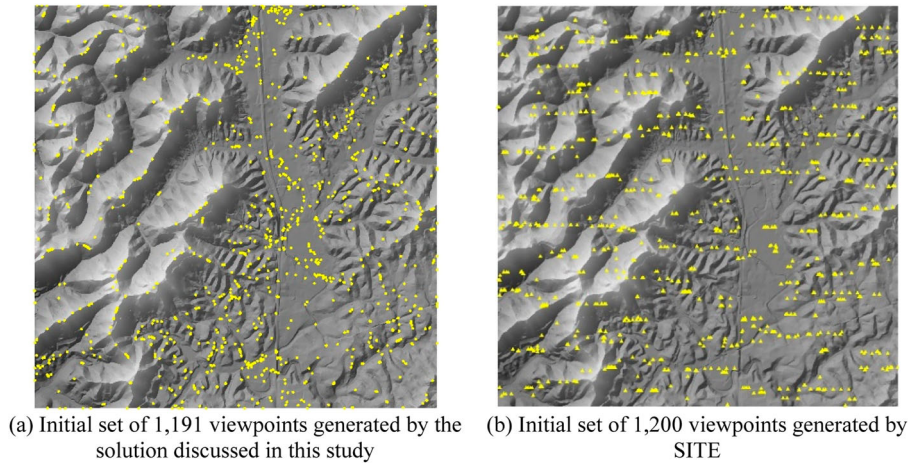


Figure 5. Initial set of viewpoints generated by different approaches. (a) Initial set of 1191 viewpoints generated by the solution discussed in this study. (b) Initial set of 1200 viewpoints generated by SITE.

When the maximum coverage of the joint viewshed is not 100% same as that generated from the initial set of viewpoints, the minimum set could be generated with different combinations of viewpoints covering the same percentage of the DEM surface. Figure 7 displays two minimum sets of viewpoints that cover 90% of the DEM. Figure 7(a) was generated first. Then all these 45 viewpoints in Figure 7(a) are removed from the initial set to generate another minimum set with 55 viewpoints in Figure 7(b). This implies that Figure 7(a) contains more optimal viewpoints. If not all of these points are removed, there should be more combination of viewpoints that can cover the same ratio of the DEM surface. Fortunately, the proposed solution can generate a minimum set of viewpoints for maximum coverage of a user-specified area of interest.

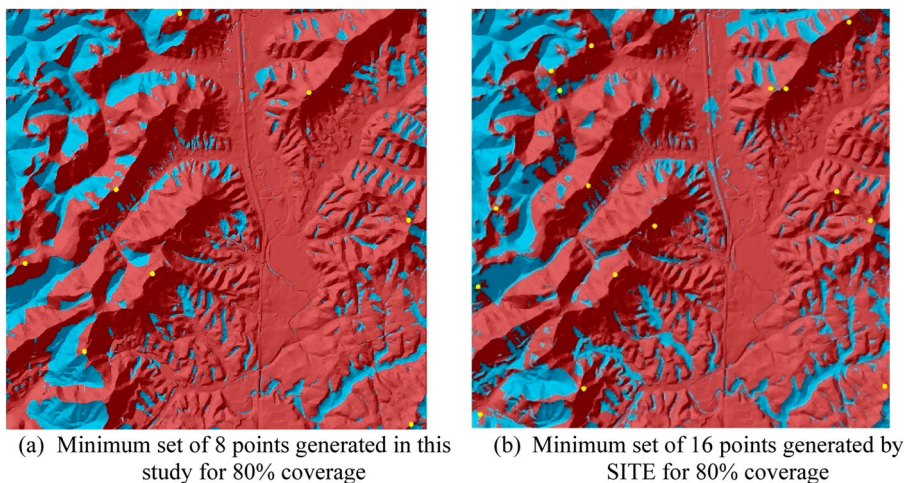


Figure 6. Minimum set of viewpoints generated by different approaches. (a) Minimum set of 8 points generated in this study for 80% coverage. (b) Minimum set of 16 points generated by SITE for 80% coverage.

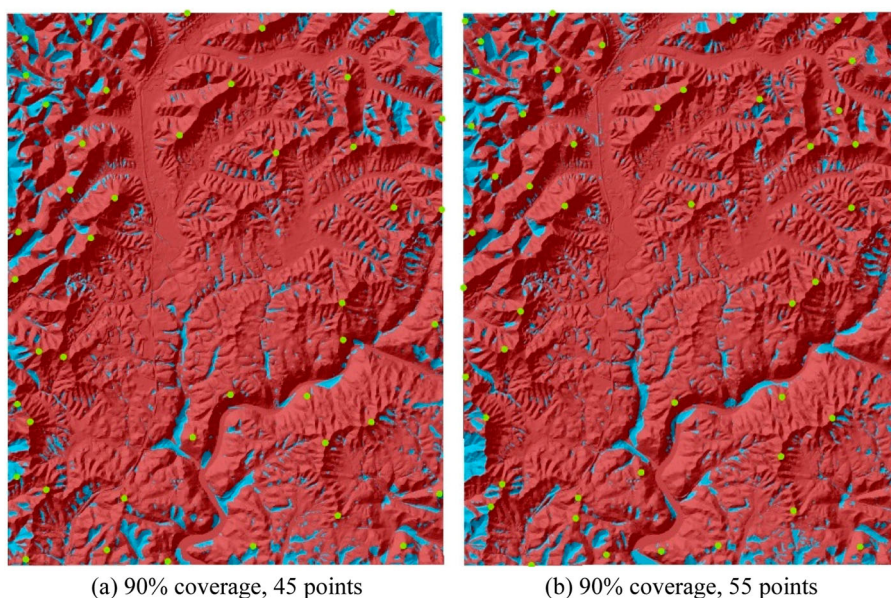


Figure 7. Minimum set of viewpoints with different compositions. (a) 90% coverage, 45 points. (b) 90% coverage, 55 points.

7. Conclusion

While a few relevant works were conducted in the past couple of decades, we resume this challenging research on generating a minimum set of viewpoints for maximum coverage over large-scale digital terrain data. The comprehensive workflow includes three steps of calculation and has been implemented and validated with satisfactory results in comparison to the alternative solution of SITE and to the current locations of FCC antennas. The computational bottleneck of the proposed workflow mainly lies in viewshed and joint viewshed calculation, counting visible pixels, and computing the ratio of overlaid viewshed. Although MPI is employed in this study to reduce the computational time, modern accelerator technologies, such as GPU or Intel's Many-integrated Core (MIC), have to be employed in the future to achieve better efficiency and scalability when a large volume of high-resolution DEM data are to be processed.

Considering that different combinations of offset heights can be applied on both the observer side and the target side, such a computational solution may require a huge storage space and memory consumption when large volumes of DEM data sets are processed. For example, 4106 initial viewpoints are automatically derived from the Summersville DEM. Each viewshed grid has 16,866,396 cells and thus needs approximately 16 MB even when using a bitmap to reduce the memory consumption by 8 times in comparison to the solution using a BIL format. More than 64 GB memory is required when all 4106 viewshed data are applied in the minimum set calculation. If different offset heights are applied, for each DEM grid, multiple viewshed grids have to be generated. In the case of the high-resolution DEM of West Virginia, if all 495 tiles of the DEM are to be processed, terabytes of storage and memory are required. In conclusion, a supercomputer system has to be deployed to support the real-world practice of such data and compute intensive application.

Disclosure statement

No potential conflict of interest was reported by the authors.

Funding

This study was partially supported by the National Science Foundation [NSF SMA-1416509].

References

- Bao, S., N. Xiao, Z. Lai, H. Zhan, and C. Kim. 2015. "Optimizing Watchtower Locations for Forest Fire Monitoring using Location Models." *Fire Safety Journal* 71: 100–109.
- De Florian, L., and P. Magillo. 2003. "Algorithms for Visibility Computation on Terrains: A Survey." *Environment and Planning B: Planning and Design* 30: 709–728. doi:10.1068/b12979.
- Feige, U. 1998. "A Threshold of $\ln n$ for Approximating Set Cover." *Journal of the ACM (JACM)* 45 (4): 634–652.
- Ferreira, C. R., M. V. Andrade, S. V. Magalhães, W. R. Franklin, and G. C. Pena. 2014. "A Parallel Algorithm for Viewshed Computation on Grid Terrains." *Journal of Information and Data Management* 5 (2): 171–180.
- Franklin, W. R. 2002. "Siting Observers on Terrain." In *Advances in Spatial Data Handling: 10th International Symposium on Spatial Data Handling*, edited by D. Richardson and P. van Oosterom, 109–120. Berlin: Springer-Verlag.
- Franklin, W. R., and C. Vogt. 2004. "Efficient Observer Siting on Large Terrain Cells (Extended Abstract)." In *GIScience 2004: Third International Conference on Geographic Information Science*, 20–23. U Maryland College Park.
- Goodchild, M. F., H. Guo, A. Annoni, L. Bian, K. De Bie, F. Campbell, M. Craglia, et al. 2012. "Next-Generation Digital Earth." *Proceedings of the National Academy of Science* 109 (28): 11088–11094.
- Goodchild, M. F., and J. Lee. 1989. Coverage Problems and Visibility Regions on Topographic Surfaces. *Annals of Operations Research* 18 (1–4): 175–186.
- Grossner, K., M. F. Goodchild, and K. Clarke. 2008. "Defining a Digital Earth system." *Transactions in GIS* 12 (1): 145–160.
- Kim, Y. H., S. Rana, and S. Wise. 2004. "Exploring Multiple Viewshed Analysis using Terrain Features and Optimisation Techniques." *Computers & Geosciences*, 30 (9): 1019–1032.
- Lee, J. 1991. "Analyses of Visibility Sites on Topographic Surface." *International Journal of Geographical Information Sciences* 5 (4): 413–429.
- Leeuwen, J. Van. 1990. *Handbook of Theoretical Computer Science: Algorithms and Complexity* (Vol. 1). Amsterdam, The Netherlands: Elsevier/Cambridge, MA: The MIT Press.
- Leiserson, C. E., R. L. Rivest, C. Stein, and T. H. Cormen. 2001. *Introduction to Algorithms*. Cambridge, MA: The MIT Press.
- Magalhaes, S. V. G., M. V. A. Andrade, and W. R. Franklin. 2010. "An Optimization Heuristic for Siting Observers in Huge Terrains Stored in External Memory." In *Hybrid Intelligent Systems (HIS), 2010 10th International Conference on*, 135–140. IEEE.
- Magalhaes, S. V. G., M. V. A. Andrade, and C. Ferreira. 2010. "Heuristics to Site Observers in a Terrain Represented by a Digital Elevation Matrix." In *GeoInfo*, 110–121.
- Magalhaes, S. V. G., M. V. A. Andrade, and R. S. Ferreira. 2011. "Using gpu to Accelerate Heuristics to Site Observers in dem Terrains." In *IADIS Applied Computing (AC 2011)*, 127–133. Rio de Janeiro.
- Planchon, O., and F. Darboux. (2002). "A Fast, Simple and Versatile Algorithm to Fill the Depressions of Digital Elevation Models." *Catena* 46 (2): 159–176.
- Pompa-García, M., R. Solís-Moreno, E. Rodríguez-Téllez, A. Pinedo-Álvarez, D. Avila-Flores, C. Hernández-Díaz, and E. Velasco-Bautista. 2010. "Viewshed Analysis for Improving the Effectiveness of Watchtowers, in the North of Mexico." *The Open Forest Science Journal* 2010 (3): 17–22.
- Sakellariou, S., S. Tampekis, F. Samara, O. Christopoulou, and A. Sfougaris. 2015. "Observatories Establishment for the Prevention of Forest Fires. The Case of Thasos Island, Greece." *International Journal of Innovative Technology and Exploring Engineering (IJITEE)*. 5 (7): 4–12.
- Van Kreveld, M. 1996. "Variations on Sweep Algorithms: Efficient Computation of Extended Viewsheds and Class Intervals." In *Proceedings of the Symposium on Spatial Data Handling '96*. pp 13A.15–13A.27.
- Warn, S. 2011. "High Performance Geospatial Analysis on Emerging Parallel Architectures." Dissertation. University of Arkansas.
- Zhao, Y., A. Padmanabhan, and S. Wang. 2013. "A Parallel Computing Approach to Viewshed Analysis of Large Terrain Data Using Graphics Processing Units." *International Journal of Geographical Information Science* 27 (2): 363–384.